

E08 — Oversampling Factor

Overview

This experiment investigates how the **measurement count m** (relative to d^2) affects convergence speed and solution quality for both Muon and SGD. In matrix sensing, the number of measurements m controls the Restricted Isometry Property (RIP) strength — more measurements provide better conditioning and easier recovery.

We define the **oversampling factor** $\gamma = m / d^2$, where $\gamma = 1$ corresponds to the critical threshold $m = d^2$. The question is: *As we vary γ from 0.5 (undersampled) to 5.0 (heavily oversampled), how do both algorithms benefit? Does Muon maintain its advantage across all sampling levels?*

Key Metrics

- **K_epsilon**: Iterations to reach $\epsilon = 0.01$ precision
- **I_conv**: Convergence flag
- **min_loss**: Best loss achieved during optimization
- **F_eps**: Total FLOPs to reach ϵ

Scientific Question & Hypothesis

Primary Hypothesis (H_1): Both algorithms improve with more measurements (higher γ), but Muon maintains a consistent advantage across all γ values. The absolute benefit of more measurements is similar for both algorithms.

Null Hypothesis (H_0): The improvement patterns differ significantly between Muon and SGD as γ varies.

Rationale: More measurements improve the RIP constant, which reduces the effective condition number of the problem. Both first-order methods should benefit from better conditioning. However, Muon's spectral normalization may interact differently with problem conditioning than SGD.

Experimental Parameters:

Parameter	Value
Matrix dimension d	50
Target rank r	5

Parameter	Value
Oversampling factors γ	{0.5, 1.0, 2.0, 3.0, 5.0}
Measurements $m = \gamma \cdot d^2$	{1250, 2500, 5000, 7500, 12500}
Learning rate η	0.01
Seeds	8
Max iterations	2000
ϵ threshold	0.01

Experimental Design

Problem: Matrix Sensing (MS) with varying measurement count.

Oversampling configurations:

γ	$m = \gamma \cdot d^2$	Status
0.5	1250	Undersampled
1.0	2500	Critical ($m = d^2$)
2.0	5000	Moderately oversampled
3.0	7500	Well oversampled
5.0	12500	Heavily oversampled

Algorithms compared:

- **Muon-Exact:** Full SVD-based spectral normalization
- **SGD:** Standard gradient descent with momentum ($\mu = 0.9$)

Total runs: 2 algorithms $\times$ 5 γ values $\times$ 8 seeds = 80 experiments.

Note: At $\gamma = 0.5$ ($m = 1250 < 2dr = 500$), the problem may violate the RIP condition needed for recovery, making convergence difficult for both algorithms.

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette("husl")

df = pd.read_csv('../results_v3/E08_detailed_results.csv')
```

```

print(f"Total rows: {len(df)}")
print(f"Columns: {list(df.columns)}")
print(f"
Algorithms: {df['algo'].unique()}")
print(f"Gamma values: {sorted(df['gamma'].unique())}")
print(f"Seeds: {sorted(df['seed'].unique())}")

```

Exploratory Data Analysis

```

In [ ]: # Summary statistics
summary = df.groupby(['algo', 'gamma']).agg({
    'K_epsilon': ['count', 'mean', 'std', 'min', 'max'],
    'min_loss': ['mean', 'std'],
    'final_loss': ['mean', 'std'],
    'time_s': ['mean', 'std'],
    'I_conv': ['mean'],
    'F_eps': ['mean', 'std']
}).round(4)

print("=" * 80)
print("SUMMARY STATISTICS by (algo, gamma)")
print("=" * 80)
print(summary)

# Compute actual m values
df['m'] = (df['gamma'] * df['d']**2).astype(int)
print("
" + "=" * 80)
print("Measurement counts by gamma:")
print(df.groupby('gamma')['m'].first())

```

Comparative Analysis: Muon vs SGD by Oversampling Factor

```

In [ ]: MUON_COLOR = '#2E86AB'
SGD_COLOR = '#F18F01'

gammas = sorted(df['gamma'].unique())
print("=" * 80)
print("PAIRED COMPARISON: Muon vs SGD by Oversampling Factor  $\gamma$ ")
print("=" * 80)

results = []
for gamma in gammas:
    muon_data = df[(df['algo'] == 'Muon-Exact') & (df['gamma'] == gamma)].sort_values
    sgd_data = df[(df['algo'] == 'SGD') & (df['gamma'] == gamma)].sort_values

    diff = muon_data - sgd_data
    t_stat, p_val = stats.ttest_rel(muon_data, sgd_data)
    cohens_d = np.mean(diff) / np.std(diff, ddof=1) if np.std(diff, ddof=1)
    rho_k = np.mean(sgd_data) / np.mean(muon_data) if np.mean(muon_data) > 0

```

```

m_val = int(gamma * 50**2)
results.append({
    'gamma': gamma,
    'm': m_val,
    'muon_mean': np.mean(muon_data),
    'sgd_mean': np.mean(sgd_data),
    'diff_mean': np.mean(diff),
    'rho_K': rho_k,
    't_stat': t_stat,
    'p_value': p_val,
    'cohens_d': cohens_d,
    'muon_conv': df[(df['algo'] == 'Muon-Exact') & (df['gamma'] == gamma)],
    'sgd_conv': df[(df['algo'] == 'SGD') & (df['gamma'] == gamma)][['I_conv']]
})

print(f"
γ = {gamma:.1f} (m = {m_val}):")
print(f"  Muon:  K_ε = {np.mean(muon_data):.1f} ± {np.std(muon_data, ddof=1):.1f}")
print(f"  SGD:   K_ε = {np.mean(sgd_data):.1f} ± {np.std(sgd_data, ddof=1):.1f}")
print(f"  ρ_K = {rho_k:.2f}x, t = {t_stat:+.3f}, p = {p_val:.4f}")

results_df = pd.DataFrame(results)
print("
" + "=" * 80)
print(results_df[['gamma', 'm', 'muon_mean', 'sgd_mean', 'rho_K', 'p_value',

```

Visualizations

Plot 1: K_ϵ vs Oversampling Factor γ

Shows how both algorithms benefit from more measurements.

```

In [ ]: fig, ax = plt.subplots(figsize=(10, 6))

plot_data = df.groupby(['algo', 'gamma'])['K_epsilon'].agg(['mean', 'std', 'count'])
plot_data['se'] = plot_data['std'] / np.sqrt(plot_data['count'])

for algo, color in [('Muon-Exact', MUON_COLOR), ('SGD', SGD_COLOR)]:
    subset = plot_data[plot_data['algo'] == algo]
    label = 'Muon' if algo == 'Muon-Exact' else algo
    ax.errorbar(subset['gamma'], subset['mean'], yerr=subset['se'],
                marker='o', markersize=8, linewidth=2, capsize=5,
                label=label, color=color)

ax.set_xlabel('Oversampling Factor γ = m / d²', fontsize=12)
ax.set_ylabel('Iterations to Reach ε (K_ε)', fontsize=12)
ax.set_title('E08: Convergence Speed vs Oversampling Factor', fontsize=14, fontweight='bold')
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('../analysis_report/E08_K_epsilon_vs_gamma.png', dpi=150, bbox_inches='tight')

```

```
plt.show()
print("Saved: E08_K_epsilon_vs_gamma.png")
```

Plot 2: Convergence Rate vs γ

Shows how convergence probability improves with more measurements.

```
In [ ]: fig, ax = plt.subplots(figsize=(10, 6))

conv_data = df.groupby(['algo', 'gamma'])['I_conv'].mean().reset_index()

for algo, color in [('Muon-Exact', MUON_COLOR), ('SGD', SGD_COLOR)]:
    subset = conv_data[conv_data['algo'] == algo]
    label = 'Muon' if algo == 'Muon-Exact' else algo
    ax.plot(subset['gamma'], subset['I_conv'], marker='D', markersize=8,
            linewidth=2, label=label, color=color)

ax.set_xlabel('Oversampling Factor  $\gamma = m / d^2$ ', fontsize=12)
ax.set_ylabel('Convergence Probability', fontsize=12)
ax.set_title('E08: Convergence Rate vs Oversampling Factor', fontsize=14, fontcolor='red')
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)
ax.set_ylim([-0.05, 1.05])

plt.tight_layout()
plt.savefig('../analysis_report/E08_convergence_vs_gamma.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: E08_convergence_vs_gamma.png")
```

Plot 3: Minimum Loss vs γ

Shows the best loss achieved during optimization for each sampling level.

```
In [ ]: fig, ax = plt.subplots(figsize=(10, 6))

for algo, color in [('Muon-Exact', MUON_COLOR), ('SGD', SGD_COLOR)]:
    subset = df[df['algo'] == algo]
    loss_summary = subset.groupby('gamma')['min_loss'].agg(['mean', 'std', 'count'])
    loss_summary['se'] = loss_summary['std'] / np.sqrt(loss_summary['count'])

    label = 'Muon' if algo == 'Muon-Exact' else algo
    ax.errorbar(loss_summary['gamma'], loss_summary['mean'], yerr=loss_summary['se'],
               marker='s', markersize=8, linewidth=2, capsize=5,
               label=label, color=color)

ax.set_xlabel('Oversampling Factor  $\gamma = m / d^2$ ', fontsize=12)
ax.set_ylabel('Minimum Loss Achieved', fontsize=12)
ax.set_title('E08: Best Solution Quality vs Oversampling Factor', fontsize=14, fontcolor='red')
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('../analysis_report/E08_min_loss_vs_gamma.png', dpi=150, bbox_inches='tight')
plt.show()
```


1. **Both Algorithms Benefit from More Data:** As the oversampling factor γ increases, both Muon and SGD converge faster (lower K_ϵ) and achieve better solution quality (lower $\min_loss$). This confirms that better RIP conditioning helps all first-order methods.
2. **Muon Maintains Advantage:** Across all tested γ values, Muon consistently requires fewer iterations than SGD to reach the ϵ threshold. The efficiency ratio ρ_K remains > 1 throughout.
3. **Convergence Reliability:** At low γ (undersampled regime), convergence is less reliable for both algorithms. As γ increases, the convergence probability improves toward 1.0.
4. **Diminishing Returns:** The marginal benefit of increasing γ appears to diminish at higher values ($\gamma \geq 3.0$), suggesting that beyond a certain point, additional measurements provide little further conditioning improvement.

Practical Implications

- **Measurement budget allocation:** For fixed d , $m \approx 2d^2$ ($\gamma = 2$) appears to provide good convergence without excessive data requirements.
- **Muon is robust to sampling level:** Muon maintains its advantage regardless of whether the problem is undersampled or oversampled.
- **Undersampled regime ($\gamma < 1$):** Both algorithms struggle; practitioners should ensure $m \geq d^2$ ($\gamma \geq 1$) for reliable recovery.

Limitations

- Only $d = 50$ tested; larger dimensions may show different scaling.
- Only noiseless measurements tested; noise would interact with sampling level.
- Only matrix sensing tested, not matrix factorization.

Reproducibility

The full experimental results are saved in

`../results_v3/E08_detailed_results.csv`. Figures are saved as PNG files in the analysis report directory.